

CS339: Abstractions and Paradigms for Programming

ADTs and Type Classes

Manas Thakur
CSE, IIT Bombay



Autumn 2025

Types Types Types

- What is a type?

Classify values.

- What is it useful for?

- What is type checking?

Ensure type correctness

- Difference between a strongly and a weakly typed language?

Strongly typed doesn't allow implicit or explicit type conversion while weakly type does.

- Why shouldn't we always use the most general type?

Operations not supported for general type, hence put constraint

- Static vs dynamic typing?

Static typing, known at compile time and Dynamic typing, checked at runtime

- Untyped languages?

Everything is treated as a bitstring. i.e. assembly

- Haskell tries to offer the best of all worlds: **Type inference!**

Compiler tries to automatically deduce the types.

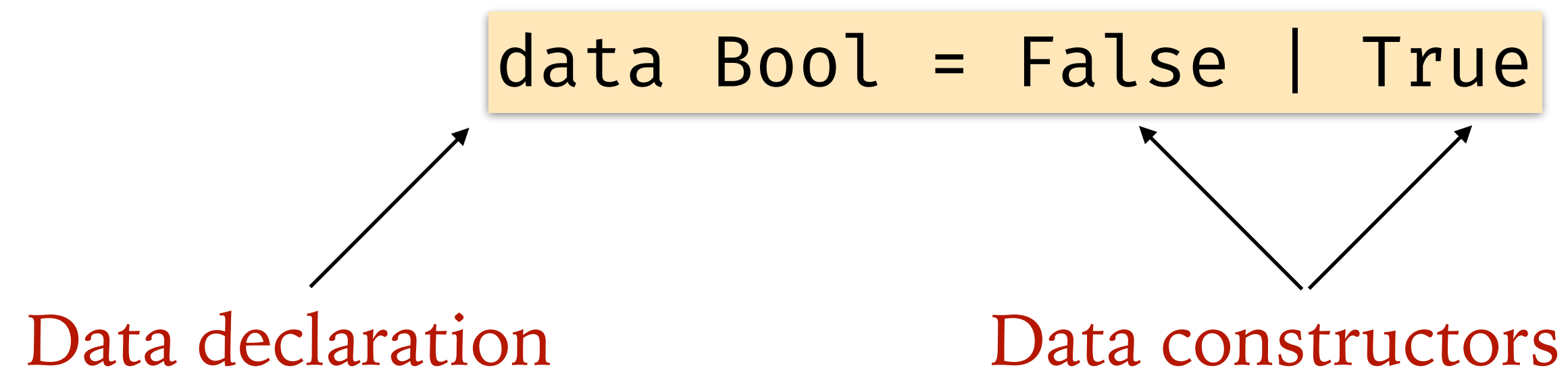
Duck typing!

If it walks like a duck
and it quacks like a duck,
then it must be a duck.



Algebraic Data Types (ADTs)

- Types can either be combinations as well as choices.
- Definition of `Bool` in Haskell:



- This is a *sum type*; it presents two *choices* for constructing a `Bool` value.

Examples of ADTs

```
data Move = North | South | East | West
```

 Named constructors

- What are constructors in OO languages?
- Check the types of North, South, East and West.
- Now we can use these types in functions:

```
type Pos = (Int,Int)
move :: Move -> Pos -> Pos
move North (x,y) = (x,y+1)
move South (x,y) = (x,y-1)
move East (x,y) = (x+1,y)
move West (x,y) = (x-1,y)
```

```
moves :: [Move] -> Pos -> Pos
moves [] p = p
moves (m:ms) p = moves ms (move m p)
```



Algebraic Data Types (ADTs)

- Types can either be combinations as well as choices.
- Definition of a Point data type:

```
data Point = Point Int Int
```

- This is a *product type*; it is constructed by *combining* two Int values.
- We can also use *sum* and *product* characteristics together:

```
data Shape = Circle Float  
           | Rectangle Float Float
```

- Do you understand the term *algebraic data types* now?



Pattern Matching with Types

```
data Shape = Circle Float  
           | Rectangle Float Float
```

- These constructors are essentially functions, which return a value of type Shape. Check their types again!
- We can again define functions using them:

Notice
pattern matching!

```
area :: Shape -> Float  
area (Circle r) = pi * r * r  
area (Rectangle ...
```

- Why won't `area :: Circle -> Float` work?
Because Shape is a type and Circle is just a constructor, Circle is not a type itself but it's a function from Float to Shape
 - Same reason why `foo :: True -> Int` won't work!



Type Parameters

```
data Maybe a = Nothing | Just a
```

Type parameter

- Maybe is a very useful type for handling failures.

```
head :: [a] -> a  
head [] = error "Empty list"  
head (x:_) = x
```

Can't return this!

```
safeHead :: [a] -> Maybe a  
safeHead [] = Nothing  
safeHead (x:_) = Just x
```

```
safediv :: Int -> Int -> Maybe Int  
safediv _ 0 = Nothing  
safediv m n = Just (m `div` n)
```

